# FinView — Guía de Administración

# Guía de administración — FinView

## 1. Stack de producción

| Componente | Tecnología | URL / Recurso |
|---|---|---|
| Frontend | Next.js 15 + Vercel | https://fin-view-topaz.vercel.app |
| Backend | Python 3.12 + FastAPI + Cloud Run (europe-west1) | finview-api-748911075551.europe-west1.run.app |
| Base de datos | Neon Postgres (eu-central-1) | ep-mute-rain-alwkqvfz |
| CI/CD | Cloud Build (trigger automático en push a main) | Proyecto: finview-brais-2026 |
| Auth | NextAuth 5.0 (JWT) | — |
| Pagos | Stripe | — |
| IA | OpenRouter ( `FINVIEW_OPENROUTER_MODEL` , default `deepseek/deepseek-v4-flash` ) | Un modelo compartido: análisis de opciones, analyst de Ideas, scorer de zonas |
| Push | ntfy.sh | — |

### Notas críticas de producción

- `BACKEND_URL` en Vercel es build-time: **redeploy manual** si cambia la URL del backend.
- `AUTH_SECRET` en Vercel debe ser idéntico a `FINVIEW_AUTH_SECRET` en Cloud Run.

- Cloud Build solo se dispara si hay cambios en: `backend/`, `Dockerfile`, `pyproject.toml`, `cloudbuild.yaml`.
- El FIFO cruzado se recalcula desde el primer año afectado en cada import, replace, delete y mutación manual — nunca limitarlo al año nominal del documento.
- Los resultados FIFO anteriores a la versión 2 se conservan en BD pero los endpoints fiscales los ignoran hasta que el ejercicio se recalcule.

---

## 2. Desarrollo local

### Requisitos

- Python 3.12+
- Node.js 20+ / pnpm 10+
- ~200 MB disco

### Instalación

```
git clone <repo-url> FinView && cd FinView

# Backend
python -m venv .venv
source .venv/bin/activate
pip install -e ".[dev]"

# Frontend
cd frontend && pnpm install && cd ..
```

### Ejecución

```
# Backend (puerto 8000, hot-reload)
uvicorn backend.app:app --reload --port 8000
```

```
# Frontend (puerto 3000, turbopack)
cd frontend && pnpm dev
```

La base de datos SQLite se crea automáticamente en `data/finview.db` al arrancar el backend.

---

## 3. Variables de entorno

### Backend (Cloud Run / `.env` local)

| Variable | Default | Descripción |
|---|---|---|
| `FINVIEW_DATABASE_URL` | `sqlite:///finview.db` | URL de la base de datos |
| `FINVIEW_AUTH_SECRET` | `change-me` | Debe coincidir con `AUTH_SECRET` del frontend |
| `FINVIEW_SECRET_KEY` | `change-me` | JWT signing key |
| `FINVIEW_JWT_EXPIRE_MINUTES` | `10080` (7 días) | Expiración del token JWT |
| `FINVIEW_STRIPE_SECRET_KEY` | — | Clave secreta Stripe |
| `FINVIEW_STRIPE_WEBHOOK_SECRET` | — | Webhook Stripe |
| `FINVIEW_STRIPE_PRICE_PRO` | — | Price ID plan Pro |
| `FINVIEW_STRIPE_PRICE_UNLIMITED` | — | Price ID plan Unlimited |
| `FINVIEW_FRONTEND_URL` | `http://localhost:3000` | URL del frontend (para redirects Stripe) |

| Variable | Default | Descripción |
|---|---|---|
| `FINVIEW_OPENROUTER_API_KEY` | — | Clave OpenRouter (opcional — activa el análisis IA de opciones, el analyst de Ideas y el scorer de zonas) |
| `FINVIEW_OPENROUTER_MODEL` | `deepseek/deepseek-v4-flash` | Modelo compartido por los tres |
| `FINVIEW_CRON_SECRET` | — | **Obligatoria en prod.** Secreto de Cloud Scheduler; sin ella los pipelines no corren |
| `FINVIEW_FRED_API_KEY` | — | Tipo libre de riesgo por divisa + régimen macro (opcional, gratuita) |
| `FINVIEW_NTFY_TOPIC` | — | Topic ntfy.sh para push notifications |
| `FINVIEW_NOTIFY_EMAIL_TO` | — | Email destino de alertas |
| `FINVIEW_NOTIFY_EMAIL_FROM` | — | Email origen (Gmail) |

| Variable | Default | Descripción |
|---|---|---|
| `FINVIEW_NOTIFY_EMAIL_PASSWORD` | — | App password de Gmail |
| `FINVIEW_OPTIONS_MONITOR_INTERVAL_MINUTES` | `15` | Frecuencia del agente de opciones |

### Frontend (Vercel / `.env.local`)

| Variable | Descripción |
|---|---|
| `NEXTAUTH_URL` | URL pública del frontend |
| `AUTH_SECRET` | Debe coincidir con `FINVIEW_AUTH_SECRET` |
| `BACKEND_URL` | URL base del backend (build-time) |
| `NEXT_PUBLIC_STRIPE_PRICE_PRO` | Price ID Stripe plan Pro |
| `NEXT_PUBLIC_STRIPE_PRICE_UNLIMITED` | Price ID Stripe plan Unlimited |

---

## 4. Base de datos

### Esquema de tablas

| Tabla | Descripción |
|---|---|
| `users` | Usuarios registrados |
| `accounts` | Cuentas por broker (una por broker importado) |
| `import_records` | Registro de importaciones (hash SHA-256 para deduplicación); `portfolio_snapshot_complete` marca informes con snapshot de cartera autoritativo |

| Tabla | Descripción |
|---|---|
| `transactions` | Operaciones y desglose de retención: total efectivo, pago español y retención extranjera |
| `tax_lots` | Lotes FIFO por broker |
| `tax_results` | Casillas Modelo 100/714 precalculadas |
| `cross_broker_lots` | Lotes FIFO cruzados entre todos los brokers |
| `fifo_adjustments` | Diferencias FIFO broker vs. FIFO cruzado |
| `option_assignments` | Asignaciones de opciones detectadas en IBKR |
| `portfolio_holdings` | Holdings a fin de año por broker |
| `price_cache` | Cache de precios Yahoo Finance |
| `data_issues` | Avisos del supervisor de datos por importación |
| `watchlist_items` | Tickers en watchlist por usuario (monitor de opciones) |
| `options_snapshots` | Snapshots de cadena de opciones por ciclo |
| `options_alerts` | Alertas generadas por el agente de opciones |
| `unusual_activity` | Actividad inusual detectada (smart money) |
| `options_agent_logs` | Log de cada ciclo del agente |
| `zonas_clave_watchlist` | Tickers en watchlist de Zonas Clave por usuario |
| `zonas_clave_analyses` | Análisis técnico calculado por ticker (zonas, RSI, MACD, precio) |
| `alt_investments` | Posiciones de inversiones alternativas (banca privada) por usuario |
| `alt_investment_catalog` | Catálogo CNMV de IIC (1 fila por ISIN de clase) para el buscador |

| Tabla | Descripción |
| --- | --- |
| `exchange_rates` | Tipos de cambio de referencia del BCE (cache, unique por fecha+divisa) |

### Orden de borrado (FK constraints)

Al eliminar una importación ( `import_records` ), el orden de borrado debe ser: 1. `cross_broker_lots` (referencia a `transactions.id` ) 2. `tax_lots` (referencia a `transactions.id` ) 3. `transactions` 4. `tax_results` 5. `fifo_adjustments` 6. `option_assignments` 7. `portfolio_holdings` 8. `data_issues` 9. `import_records`

Al eliminar una cuenta ( `accounts` ) se eliminan en cascada todas sus transacciones y holdings asociados.

### Reset de la DB (local)

```
rm data/finview.db
# Reiniciar el backend — se recrea automáticamente
```

---

## 5. Supervisor de datos (import agent)

`backend/services/data_supervisor.py` ejecuta checks de calidad tras cada import.
Se invoca automáticamente al final de cada importación de cualquier broker.

### Checks y estrategia de resolución

| Check | Detección | Resolución automática |
| --- | --- | --- |
| `rights_as_sell` | SELL cuyo nombre contiene “derechos/rights/drets” | Reclasifica a DIVIDEND, qty = 0 |
| `missing_buys` | Holdings sin transacciones de | Aviso `needs_input` |

| Check | Detección | Resolución automática |
|---|---|---|
| | compra | |
| `holdings_mismatch` | Qty en broker ≠ qty calculada de transacciones | Aviso `needs_input`; si coincide con un split conocido, se resuelve en `potential_split` |
| `missing_ticker` | ISIN sin ticker en Yahoo Finance | Búsqueda automática en yfinance; si falla, aviso `needs_input` |
| `potential_split` | Split detectado vía yfinance con ratio fuera de [0.9, 1.1] | **Tres casos:** |
| | → Mismo ticker, múltiples ISINs con compras en ambos lados | `auto_resolved` — datos ya correctos |
| | → Mismo ISIN, qty × ratio ≈ holdings (±5%) | `auto_resolved` — ajusta qty y precio en las transacciones previas |
| | → No se puede confirmar | `needs_input` — aviso con datos para revisión manual |
| `price_sanity` | BUY/SELL con precio = 0 | Scrip dividends: `auto_resolved`; otros: `needs_input` |

### Estados de un `DataIssue`

| Estado | Significado |
|---|---|
| `auto_resolved` | El supervisor lo corrigió automáticamente; visible como informativo en la UI |
| `needs_input` | Requiere revisión manual; aparece como aviso amarillo/rojo tras la importación |
| `dismissed` | El usuario lo descartó manualmente |

### Flujo del supervisor

```
Import → parse → save transactions/holdings/casillas
    └→ recalculate_cross_broker_fifo()
    └→ _run_supervisor(db, import_id)
            ├─ _check_rights_as_sells()      → auto-fix
            ├─ _check_missing_buys()         → flag
            ├─ _check_holdings_mismatch()    → flag
            ├─ _check_missing_ticker()       → auto-resolve / flag
            ├─ _check_split_candidates()     → auto-resolve / auto-fix / flag
            └─ _check_price_sanity()         → auto-resolve / flag
    └→ db.commit()
    └→ return ImportResult { issues: [...] }
```

### Lógica de deduplicación de issues

Un `DataIssue` con `status=needs_input` no se crea dos veces para el mismo `(issue_type, isin)`. Si ya existe un aviso pendiente, la siguiente importación no lo duplica.

---

## 6. Agente de opciones

El agente ( `backend/services/options_agent.py` ) es un worker APScheduler que corre dentro del proceso FastAPI.

### Arquitectura

```
WatchlistItem (DB)
    └→ fetch_options (Yahoo Finance / MEFF)
    └→ enrich_options_with_greeks (Black-Scholes)
    └→ analyze_ticker (alertas, unusual activity, sentiment)
    └→ OptionsSnapshot, OptionsAlert, UnusualActivity (DB)
    └→ _run_claude_analysis (opcional, vía OpenRouter)
    └→ OptionsAgentLog (DB)
    └→ notify_alerts (ntfy.sh + email)
```

## Fuentes de datos

- **Yahoo Finance** ( `yfinance` ): tickers US/EU sin sufijo `.MC` .
- **MEFF** ( `requests` + `BeautifulSoup` ): tickers con sufijo `.MC` (bolsa española). Usa proxy `allorigins.win` para evitar bloqueos CORS.

## Greeks (Black-Scholes)

Implementados en `backend/services/greeks_calculator.py` sin dependencias externas (solo `math` ).
Parámetros por defecto: r = 5.3%, q = 1.3%, 252 días de trading.

## Umbrales de alerta

| Parámetro | Valor | Descripción |
|---|---|---|
| `HIGH_IV_THRESHOLD` | 50% | IV media alta |
| `HIGH_PCR_THRESHOLD` | 1.5 | P/C ratio bajista |
| `LOW_PCR_THRESHOLD` | 0.5 | P/C ratio alcista |
| `IV_SKEW_THRESHOLD` | 10 pp | Diferencia IV puts-calls |
| `IV_PREMIUM_RATIO` | 1.5× | IV vs. volatilidad histórica |
| `UNUSUAL_VOLUME_RATIO` | 3.0× | Volumen / Open Interest |

## Análisis IA (persona *VolatilityEdge*)

- Proveedor: **OpenRouter** (httpx a mano, sin SDK). Modelo: `settings.openrouter_model` , default `deepseek/deepseek-v4-flash`
- Max tokens: 1500 · sin `temperature` explícita (se usa la del modelo)
- Activación: solo si `FINVIEW_OPENROUTER_API_KEY` está configurado **Y** existen alertas en el ciclo
- Output: markdown en español, exactamente 7 secciones, truncado a 5000 caracteres en DB
- **Fire-and-forget**: que la IA no esté disponible nunca puede romper el ciclo

## Endpoints de control

| Método | Endpoint | Descripción |
|---|---|---|
| `POST` | `/options/agent/run` | Ciclo manual inmediato |
| `GET` | `/options/agent/status` | Estado del scheduler, último ciclo, total ciclos |
| `POST` | `/options/agent/start` | Arrancar job periódico APScheduler |
| `POST` | `/options/agent/stop` | Parar job periódico |

### Cascade al eliminar un ticker de cartera

Al eliminar un ticker de la cartera de opciones (posiciones IBKR), se eliminan en cascada sus entradas en `watchlist_items`, `options_alerts` y `unusual_activity`.

---

## 7. Motor FIFO cruzado

El motor en `backend/services/fifo_engine.py`:

1. Detecta ISIN no vacíos vendidos en el ejercicio y comprados en dos o más cuentas del usuario.
2. Carga todas las compras y ventas reales hasta el 31 de diciembre del ejercicio; las ventas históricas consumen inventario antes de procesar las del año objetivo.
3. Ordena por fecha e ID y hace matching FIFO global. Una compra con fecha posterior a la venta nunca puede emparejarse.
4. Incluye `SPINOFF` como adquisición de cantidad con base desconocida. Si una venta del ejercicio alcanza esa cantidad, crea `fifo_missing_cost_basis` y descarta el resultado fiscal del ISIN; `missing_cost_basis` queda como incidencia fuente del supervisor y nunca se interpreta la base como cero. La acción `set_price` persiste el coste unitario del SPINOFF y llama a `recalculate_fifo_from_year`.
5. Si cualquier venta queda sin inventario verificable, descarta todo el resultado del ISIN: no persiste lotes parciales ni ajustes autoritativos.
6. Persiste solo los lotes vendidos en el ejercicio solicitado. El periodo de tenencia es informativo; la casilla se decide por el signo del resultado: ganancia → 332, pérdida → 337.

7. Agrega resultados por ISIN/casilla y redondea después a céntimos con `ROUND_HALF_UP`; `CrossBrokerLot` conserva seis decimales para auditoría.
8. Compara por separado ganancias y pérdidas del broker contra FIFO. Esto permite mover importes entre 332/337 aunque el P&L neto del ISIN no cambie.
9. Persiste `CrossBrokerLot` y `FIFOAdjustment` con `calculation_version="2"`. Los agregados fiscales, dashboard, exportación y UI aplican solo esa versión.

`recalculate_fifo_from_year` se invoca con la fecha más antigua afectada y recorre ese año y todos los posteriores con ventas o ajustes existentes. Esto cubre importación, reemplazo, borrado de archivo/cuenta y CRUD manual, incluida la limpieza del último sell. En PostgreSQL, un advisory lock transaccional por usuario evita dos rebuilds FIFO simultáneos. Otro lock por usuario, broker y ejercicio cubre desde la comprobación de duplicados hasta el commit del reemplazo anual; no requiere migración. Las auto-correcciones del supervisor y las resoluciones manuales que cambian historia recalculan desde el primer año alterado antes de confirmar la transacción.

### Migración y datos históricos

`cross_broker_lots.calculation_version` y los cinco campos de desglose/versionado de `fifo_adjustments` se añaden mediante `_ADDITIVE_COLUMNS` / `ensure_columns()` al arrancar. Son nullable para mantener compatibilidad con filas antiguas.

No hay backfill automático: las filas legacy quedan almacenadas pero no se muestran ni alteran los totales. Para activarlas con v2 hay que reemplazar o reimportar el ejercicio. La invalidación descendente sí es automática desde el primer año afectado. Para corregir `net_amount_eur` y `TaxLot` ya persistidos con la semántica antigua de comisiones/primas, reimportar los informes DEGIRO e IBKR afectados.

---

## 8. Zonas Clave

`backend/services/zonas_clave_service.py` calcula soporte/resistencia algorítmico para cada ticker.

### Arquitectura

```
ZcWatchlistItem (DB)
    └─► yfinance.download(ticker, period="1y")
```

```
    └─► _compute_zones() → soporte/resistencia por clustering de máximos/mínimos
    └─► _compute_indicators() → RSI(14), MACD(12,26,9)
    └─► ZcAnalysis (DB) → zones, rsi, macd_signal, price_history
```

### Endpoints

| Método | Endpoint | Descripción |
|---|---|---|
| `GET` | `/zonas-clave/watchlist` | Lista de tickers en watchlist |
| `POST` | `/zonas-clave/watchlist` | Añadir ticker |
| `DELETE` | `/zonas-clave/watchlist/{id}` | Eliminar ticker |
| `GET` | `/zonas-clave/analysis/{ticker}` | Último análisis de un ticker |
| `GET` | `/zonas-clave/analysis` | Últimos análisis de todos los tickers |
| `POST` | `/zonas-clave/analyze/{ticker}` | Forzar recálculo de un ticker |
| `POST` | `/zonas-clave/analyze-all` | Recalcular todos los tickers |
| `GET` | `/zonas-clave/scheduler/status` | Estado del scheduler |
| `POST` | `/zonas-clave/scheduler/start` | Arrancar scheduler 24h |
| `POST` | `/zonas-clave/scheduler/stop` | Parar scheduler |

---

## 9. Componentes de navegación frontend

### QuickNav ( `frontend/src/components/QuickNav.tsx` )

Barra de 5 iconos de acceso rápido presente en todas las páginas internas.
Acepta la prop `current: QuickNavPage` (literal de la ruta activa) para resaltar el icono correspondiente.

```
<QuickNav current="/portfolio" />
```

Destinos fijos: `/dashboard`, `/portfolio`, `/options`, `/zonas-clave`, `/alt-investments`.
Usa `TooltipTrigger` de `@base-ui/react/tooltip` para mostrar el nombre al hover.

## Portfolio position detail

La reconciliación de cartera usa el último `ImportRecord` con `portfolio_snapshot_complete=true` por cuenta. El marcador también representa un snapshot válido sin holdings; solo se activa para el informe fiscal ING y cuando el lote BBVA incluye el documento de patrimonio. Los certificados ING de operaciones y los lotes BBVA parciales no pueden cerrar posiciones. Las importaciones legacy sin marcador conservan el fallback al último año que tenga filas en `portfolio_holdings`.

Después del snapshot se aplican BUY/SELL/SPINOFF por fecha e ID. Las ventas consumen lotes FIFO; un lote de origen desconocido propaga `total_cost_eur=NULL` hasta que se vende por completo. Los agregados nunca suman solo la parte conocida del coste ni publican un valor de mercado total si falta el precio de alguna posición.

El endpoint `GET /portfolio/positions/{isin}` devuelve ahora dos arrays separados: - `transactions`: solo BUY/SELL/CORPORATE_ACTION (sin DIVIDEND), limitado a 200 - `dividends_by_year`: dividendos agregados por año fiscal `{ year, gross_eur, net_eur, count }`

Esto evita que dividendos con fecha fiscal idéntica (p.ej. ING reporta todos los dividendos con fecha `YYYY-12-31`) aparezcan duplicados en la UI.

## Rendimiento — consolidación de ISINs

`backend/api/charts._merge_same_ticker_rows()` post-procesa el endpoint `/charts/dividend-yield`:
Cuando dos ISINs distintos mapean al mismo ticker de Yahoo Finance (split con cambio de ISIN), sus filas se fusionan sumando dividendos, coste y valor, y recalculando los ratios de rentabilidad.

---

## 10. Tests

```
# Todos los tests (estado tras integridad FIFO residual: 453 passed, 131 skipped sin
        fixtures reales)
pytest backend/tests/ -v

# Solo un módulo
pytest backend/tests/test_fifo_engine.py -v

# Con cobertura
pytest backend/tests/ --cov=backend --cov-report=term-missing
```

Los tests que necesitan archivos reales (PDFs/CSVs en `files/`) se saltan automáticamente si no existen.

---

## 11. Añadir un nuevo broker

### 11.1 Parser

Crear `backend/importers/<broker>.py`:

```
from dataclasses import dataclass, field
from pathlib import Path
from backend.importers.base import ParsedTransaction, ParsedPortfolioHolding

@dataclass
class NewBrokerReport:
    fiscal_year: int
    transactions: list[ParsedTransaction] = field(default_factory=list)
    dividends: list[ParsedTransaction] = field(default_factory=list)
    holdings: list[ParsedPortfolioHolding] = field(default_factory=list)

def parse_new_broker(file_path: Path) -> NewBrokerReport: ...
def compute_file_hash(path: Path) -> str: ...
```

### 11.2 Broker enum

En `backend/models.py`:

```
class Broker(str, Enum):
    NEW_BROKER = "NEW_BROKER"
```

### 11.3 Import service

En `backend/services/import_service.py`, añadir `import_new_broker_report()` siguiendo el patrón existente: 1. `_check_duplicate()` → hash SHA-256 2. `_check_year_duplicate()` → broker + año 3. `_get_or_create_account()` 4. Crear `ImportRecord` 5. Insertar transacciones, holdings, casillas 6. `recalculate_cross_broker_fifo(db, fiscal_year)` 7. `_run_supervisor(db, import_rec.id)` 8. `db.commit()`

### 11.4 Endpoint API

En `backend/api/imports.py`:

```
@router.post("/new-broker", response_model=ImportResult)
def upload_new_broker(file: UploadFile, db: DbSession, user: CurrentUser): ...
```

### 11.5 Frontend

En `frontend/src/lib/api.ts`:

```
export async function uploadNewBrokerReport(file: File) {
  const form = new FormData();
  form.append("file", file);
  return fetchApi("/imports/new-broker", { method: "POST", body: form });
}
```

En `frontend/src/components/FileUpload.tsx`: 1. Añadir `detectBroker()` — detección por nombre de archivo. 2. Añadir entrada en `BROKERS` con key, label, accept y color. 3. Mapear en `uploadBroker()` si no es BBVA.

---

## 12. Inversiones Alternativas y tipos de cambio

### Inversiones Alternativas ( `backend/api/alt_investments.py` )

Tracker manual de posiciones de banca privada (PE/VC/RE, Hedge Fund, Producto Estructurado, Otro). MVP sin IA ni importación.

- **Modelo `AltInvestment`** — una fila por posición, con los flujos como agregados ( `committed_capital` , `called_capital` , `distributed_capital` , `current_nav` ) más `type` , `status` , `manager` , `vintage_year` , `geography` , `currency` , `notes` . No hay sub-tabla de movimientos.
- **Métricas derivadas** (calculadas en cada respuesta, nunca almacenadas): TVPI = (distribuido + NAV) / called, DPI = distribuido / called, RVPI = NAV / called, unfunded = comprometido − called. Todas `None` cuando called = 0.
- Las tablas nuevas se crean con `Base.metadata.create_all` al arrancar (no requieren Alembic para altas).

| Método | Endpoint | Descripción |
|---|---|---|
| `GET` | `/alt-investments/` | Listar posiciones (con métricas por fila) |
| `GET` | `/alt-investments/summary` | Consolidación + asignación por tipo/geografía/divisa (ponderada por NAV) |
| `GET` | `/alt-investments/search` | Buscar/explorar el catálogo CNMV ( `q` , `type` , `sort` , `min_fund_size` , `max_fund_size` , `limit` ); incluye NAV y tamaño del fondo |
| `POST` | `/alt-investments/` | Crear posición |
| `PATCH` | `/alt-investments/{id}` | Actualizar posición |
| `DELETE` | `/alt-investments/{id}` | Eliminar posición |

> Columnas añadidas a tablas ya desplegadas ( `alt_investments.isin` ; `alt_investment_catalog.nav` , `fund_size` , `data_date` ): como `create_all` no altera tablas existentes y no hay Alembic, se registran en `database._ADDITIVE_COLUMNS` y `ensure_columns()` (llamado tras `create_all` en `app.py` ) las añade de forma idempotente al arrancar. Cualquier columna nueva sobre una tabla ya desplegada debe seguir este patrón.

## Buscador — catálogo CNMV ( `backend/services/cnmv_catalog.py` )

Da de alta fondos españoles regulados buscando por nombre o ISIN y pre-rellena el formulario (solo identidad: nombre, tipo, gestora, ISIN; importes manuales).

- **Fuente**: registro mensual gratuito "información individual de IIC" de la CNMV (XML dentro de ZIP, sin auth). Flujo: página del año → BeautifulSoup del último enlace `lnkZip` (token opaco `verdocumento/ver?e=` ) → `_download_zip_members` lee dos XML (1 reintento ante caída transitoria): **FONDREGISTRO** (identidad) → `_parse_registry` , una fila en `alt_investment_catalog` por ISIN de clase; **FONDMENS** (NAV/patrimonio) → `_parse_mens` , fusionado por ISIN.
- **Enriquecimiento**: `nav` = último VL diario ( `VL_DiaN` ) de la clase; `fund_size` = suma del último `Patrimonio_DiaN` de todas las clases del fondo; `data_date` del `FechaDatos` raíz. Columnas añadidas vía `_ADDITIVE_COLUMNS` / `ensure_columns()` (la tabla ya existe en prod).
- `refresh_iic_catalog()` hace upsert del catálogo; corre en un hilo en segundo plano al arrancar cuando `catalog_is_stale` (>7 días). Best-effort, nunca bloquea ni rompe la página, nunca por request.
- `search_catalog(db, q, type_filter, min_fund_size, max_fund_size, sort, limit)` es solo-DB: ILIKE sobre nombre + ISIN exacto, prefijos primero. `sort` ∈ {name, fund_size_desc, nav_desc, nav_asc}. **Modo explorar**: con `q` corto/vacío devuelve filas si hay filtro o `sort` no por defecto; si no, `[]` .
- **Cobertura**: el fichero mensual es solo Fondos de Inversión (Tipo=FI) — ~1.400 fondos / ~3.000 ISINs, mapeados a `OTHER` , todos con NAV + patrimonio. SICAV / IIC inmobiliaria / hedge / capital riesgo están en registros CNMV separados (diferido); `_TIPO_MAP` ya contempla FII→REAL_ESTATE, FIL→HEDGE_FUND. Las métricas de PE (DPI/TVPI/IRR/vintage/hurdle) no son públicas — manuales en la posición.

## Tipos de cambio ECB ( `backend/services/exchange_rates.py` )

Servicio de tipos de referencia oficiales del BCE como fallback IRPF cuando el archivo del broker no trae el tipo (o en entradas manuales). Portado desde OptionsAI.

- Tabla `exchange_rates` (rate_date, currency, rate, source) con unique (rate_date, currency).
- `sync_rates()` corre en un hilo en segundo plano al arrancar ( `backend/app.py` lifespan): historia completa el primer boot, luego feed de 90 días; se omite si los datos están frescos (≤1 día). **Nunca** se ejecuta dentro de una request.
- `get_rate(currency, on)` es solo-DB (sin HTTP) con fallback ±7 días para fines de semana/festivos. `to_eur(amount, currency, on)` = amount / rate. EUR siempre es 1.
- Endpoint de consulta: `GET /fiscal/fx-rate?currency=USD&on=YYYY-MM-DD` .

---

## 13. Troubleshooting

**Error 500 al borrar importación**
Causa más común: FK violation en `cross_broker_lots` . El orden de borrado en `_delete_import_data` debe eliminar CrossBrokerLots antes que Transactions.

**“duplicate import”**
El archivo ya fue importado (mismo hash SHA-256). Eliminar la importación anterior primero.

**“year already imported”**
Ya existe importación del mismo broker + año. La nueva importación reemplaza automáticamente.

**Casillas no cuadran**
Comparar el FIFO cruzado (pestaña casillas) con el del broker individual. Los ajustes FIFO explican las diferencias.

**Frontend no conecta al backend**
Verificar `BACKEND_URL` en Vercel y que CORS en `backend/app.py` incluye el dominio del frontend.

**El agente de opciones no genera análisis IA**
Verificar que `FINVIEW_OPENROUTER_API_KEY` está configurado en Cloud Run y que existe al menos un ticker en la watchlist con alertas activas (sin alertas no se llama al modelo, por diseño).

**Una sección lee vacía y no hay ningún error**
Casi siempre es que falta el job de Cloud Scheduler, no un bug de la UI. El `APScheduler` de la interfaz muere al escalar a cero. Comparar los jobs ( `gcloud scheduler jobs list --location=europe-west1` ) con los endpoints `/cron` del código. Ver deployment.md.

**MEFF no devuelve datos**

El proxy `allorigins.win` puede estar caído. El agente hace fallback a Yahoo Finance para el precio y devuelve `None` para la cadena de opciones.

**Empresa duplicada en Rendimiento (ej. Aena)**

Causa: dos ISINs para la misma empresa (pre/post-split) con dividendos en ambos.
Resolución automática: `_merge_same_ticker_rows` los consolida si ambos ISINs están en `_ISIN_TICKER_MAP` con el mismo ticker. Si el ticker no está mapeado, añadirlo en `backend/services/prices.py → _ISIN_TICKER_MAP`.

**Split no detectado automáticamente**

El check `potential_split` solo actúa sobre ISINs presentes en `portfolio_holdings`. Si el ISIN ya no tiene holdings activos, no se comprueba. Verificar manualmente en el extracto del broker.